Computability: Recitation 6

May 7, 2026

1 R, RE, coRE

1.1 Definitions and examples

Reminder:

$$\mathrm{R} \stackrel{\text{def}}{=} \{L : \text{There is a TM } M \text{ that decides } L\}$$
$$\mathrm{RE} \stackrel{\text{def}}{=} \{L : \text{There is a TM } M \text{ that recognizes } L\}$$

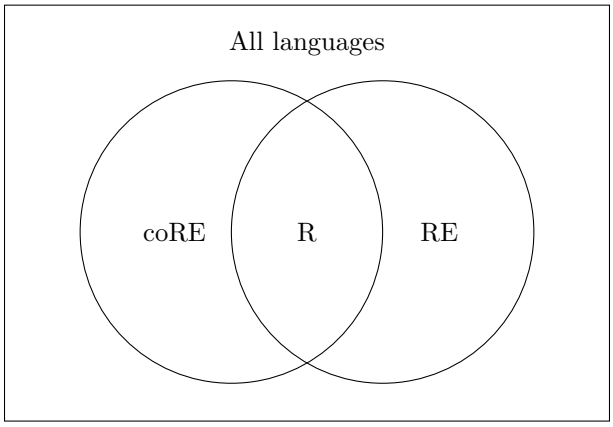
A reduction from $A \subseteq \Sigma^*$ to $B \subseteq \Sigma^*$ is a computable function $f : \Sigma^* \to \Sigma^*$ such that $x \in A \Leftrightarrow f(x) \in B$.

The class coRE is defined as follows:

$$\mathrm{coRE} \stackrel{\text{def}}{=} \left\{L : \text{There is a TM } M \text{ that recognizes } \overline{L}\right\}$$

Since $\mathrm{HALT}_{\mathrm{TM}} \in \mathrm{RE}$, we have $\overline{\mathrm{HALT}_{\mathrm{TM}}} \in \mathrm{coRE}$.

The classes $\mathrm{R}, \mathrm{RE}, \mathrm{coRE}$ can be pictured as follows:



1.2 Reduction theorem

Theorem 1 (reduction theorem)**.** *Let $L_1, L_2 \subseteq \Sigma^*$ be languages such that $L_1 \leq_m L_2$. For every class of languages $\mathcal{C} \in \{\mathrm{R}, \mathrm{RE}, \mathrm{coRE}\}$, if $L_2 \in \mathcal{C}$ then $L_1 \in \mathcal{C}$. Equivalently, if $L_1 \notin \mathcal{C}$ then $L_2 \notin \mathcal{C}$.*

Intuition: If $L_1 \leq_m L_2$ then L_2 is "at least as hard" as L_1, in the following sense: if there is something we know how to do with L_2 (decide it, or recognize it, or recognize its complement), we can do the same for L_1 by first applying the reduction. Equivalently, if there is something we know is impossible to do with L_1 (decide it, recognize it, recognize its complement), then the same applies for L_2.

Example: We saw that $\mathrm{HALT}_{\mathrm{TM}} \leq_m \mathrm{A}_{\mathrm{TM}}$. Since $\mathrm{HALT}_{\mathrm{TM}} \notin \mathrm{R}$, we have $\mathrm{A}_{\mathrm{TM}} \notin \mathrm{R}$.

Example: In the previous recitation we defined $\mathrm{ALL}_{\mathrm{TM}} \stackrel{\text{def}}{=} \{\langle M \rangle : L(M) = \Sigma^*\}$. This language is outside RE and coRE:

Proposition 2. $\mathrm{ALL}_{\mathrm{TM}} \in \overline{\mathrm{RE} \cup \mathrm{coRE}}$.

Remark: Here we use overline to denote the complement of a *collection* of languages (not of a language), with respect to the set of all languages.

Proof. We first prove: $\text{HALT}_{\text{TM}} \leq_m \text{ALL}_{\text{TM}}$. Since $\text{HALT}_{\text{TM}} \notin \text{coRE}$, it follows that $\text{ALL}_{\text{TM}} \notin \text{coRE}$. The reduction works as follows:

- Construction: Let $f(\langle M, w\rangle) = \langle M'\rangle$, where M' acts as follows on input $x \in \Sigma^*$:
  - Remove x from the tape, and write w.
  - Run M on w.
  - If M halts during the run, accept.
- Correctness:
  - Suppose $\langle M, w\rangle \in \text{HALT}_{\text{TM}}$. Then M halts on w. Therefore, given any x after running M on w, M' accepts. so $\langle M'\rangle \in \text{ALL}_{\text{TM}}$.
  - Suppose $\langle M, w\rangle \notin \text{HALT}_{\text{TM}}$. Then M doesn't halt on w. Therefore, for any x, M' does not halt and $x \notin L(M')$. Thus, $L(M') = \phi \neq \Sigma^* \implies \langle M'\rangle \notin \text{ALL}_{\text{TM}}$.
- Computability: M' deletes any input, overrides with w, and runs M on w. It is possible to encode such a machine using procedures we have seen.

Next we prove $\overline{\text{HALT}_{TM}} \leq_m \text{ALL}_{\text{TM}}$. Since $\overline{\text{HALT}_{\text{TM}}} \in \text{coRE} \setminus \text{R}$, we have $\overline{\text{HALT}_{\text{TM}}} \notin \text{RE}$, so $\text{ALL}_{\text{TM}} \notin \text{RE}$.
Construction: Let $f(\langle M, w\rangle) = \langle M'\rangle$, where M' acts as follows on input $x \in \Sigma^*$:

- Run M on w for $|x|$ steps.
- If M halts during the run, reject. Otherwise, accept.

Correctness:

- Suppose $\langle M, w\rangle \in \overline{\text{HALT}_{TM}}$. Then M does not halt on w. Therefore, after running M for $|x|$ steps, M' accepts. This is true for every x, so $\langle M'\rangle \in \text{ALL}_{\text{TM}}$.
- Suppose $\langle M, w\rangle \notin \overline{\text{HALT}_{TM}}$. Then M halts on w after some finite number of steps. Therefore, for sufficiently long input x, M' finds that M accepts w and rejects x. So $\langle M'\rangle \notin \text{ALL}_{\text{TM}}$.

Computability: M' runs M on w, with a step counter limited to $|x|$. It is possible to encode such a machine using procedures we have seen. □

Proposition 3. $\text{REG}_{\text{TM}} \stackrel{\text{def}}{=} \{\langle M\rangle : L(M) \in \text{REG}\} \in \overline{\text{RE} \cup \text{coRE}}$.

Proof. We show that $\text{REG}_{\text{TM}} \notin \text{coRE}$ by a reduction from HALT_{TM}. The proof that $\text{REG}_{\text{TM}} \notin \text{RE}$ is left as an exercise.

- Construction: Let $f(\langle M, w\rangle) = \langle M'\rangle$, where M' acts as follows on input $x \in \Sigma^*$:
  - If $x = a^n b^n$ for some $n \geq 0$, accept.
  - Run M on w. If it halts, accept.
- Correctness:
  - Suppose $\langle M, w\rangle \in \text{HALT}_{\text{TM}}$. Then M halts on w. Therefore, M' accepts every input x. So $L(M') = \Sigma^*$, which is regular, hence $\langle M'\rangle \in \text{REG}_{\text{TM}}$.
  - Suppose $\langle M, w\rangle \notin \text{HALT}_{\text{TM}}$. Then M does not halt on w. Therefore, M' accepts x if and only if it is of the form $a^n b^n$. We have seen that $\{a^n b^n : n \geq 0\}$ is not regular, so $\langle M'\rangle \notin \text{REG}_{\text{TM}}$.
- Computability: M' checks whether the input is of the form $a^n b^n$, and runs M on w. It is possible to encode such a machine using procedures we have seen.

□

1.3 Hard and complete languages

Definition 4 (hardness and completeness)**.** *For every class $\mathcal{C} \in \{\text{R}, \text{RE}, \text{coRE}\}$, a language L is called* $\mathcal{C}$-hard *if for every $K \in \mathcal{C}$ we have $K \leq_m L$. A language L is called* $\mathcal{C}$-complete *if it is $\mathcal{C}$-hard and also $L \in \mathcal{C}$.*

Intuition: We think of $\mathcal{C}$-hard as "at least as hard as every language in $\mathcal{C}$", and $\mathcal{C}$-complete as "hardest in $\mathcal{C}$".

Example: We saw in class that $\mathrm{A_{TM}}$ is RE-hard.

Proposition 5 (exercise)**.** *For a class $\mathcal{C}$ as above, if L is $\mathcal{C}$-hard and $L \leq_m K$, then K is also $\mathcal{C}$-hard.*

Proposition 6. *If L is* RE*-hard, then $L \notin$ coRE. Similarly, if L is* coRE*-hard, then $L \notin$ RE. Therefore, if L is both* RE*-hard and* coRE*-hard then $L \notin \mathrm{RE} \cup \mathrm{coRE}$.*

Proof. We prove the first statement (the second one is very similar). Let L be an RE-hard language, and suppose for contradiction that $L \in$ coRE. Since L is RE-hard and $\mathrm{HALT_{TM}} \in$ RE, we have $\mathrm{HALT_{TM}} \leq_m L$. Therefore $\overline{\mathrm{HALT_{TM}}} \leq_m \overline{L}$. But $\overline{L} \in$ RE and $\overline{\mathrm{HALT_{TM}}} \notin$ RE, contradicting the reduction theorem. □

Proposition 7. *A language L is* RE*-complete iff $\overline{L}$ is* coRE*-complete.*

Proof. Suppose that $L \in$ RE, and that for every $K \in$ RE we have $K \leq_m L$. We claim that $\overline{L}$ is coRE-complete. We have that $\overline{L} \in$ coRE because $L \in$ RE. It is left to show coRE-hardness. Let $L' \in$ coRE. We have $\overline{L'} \in$ RE, so $\overline{L'} \leq_m L$. As we saw in the exercise, this is equivalent to $L' \leq_m \overline{L}$.

The other direction is similar. □

Proposition 8. *USELESS* $= \{\langle M \rangle$: *there exists a state $q \notin \{q_{acc}, q_{rej}\}$ in M that is never reached, on any input$\}$ is* coRE*-complete*

Proof.

1. USELESS$\in$ coRE:

   $$\overline{\texttt{USELESS}} = \texttt{USEFUL} \cup \texttt{E}$$

   where E is the set of invalid inputs. We show that there exists a TM T such that $L(T) =$ USEFUL that is, USEFUL $\in$ RE. Given input $\langle M \rangle$, a TM T can simulate M on every input incrementally like we did with NON-EMPTY, while keeping track of visited states. If, at any point, every state was visited, then $\langle M \rangle \in$ USEFUL and we accept. Otherwise, we do not halt.
   We want to show that $L(T) = \overline{\texttt{USEFUL}}$

   - $\langle M \rangle \notin$ USEFUL then there exists a state q which is never reached for a run of M over any word w. Thus, T does not halt, and does not accept $\langle M \rangle$, so $\langle M \rangle \notin L(T)$
   - If $\langle M \rangle \in$ USEFUL, then every state q in Q_M can be reached. For each state q, there exists some input w and a step number $n \geq 0$ where M, running on w, reaches q. Since M has only finitely many states, there are finitely many such words and step numbers. We choose the longest word and the largest step number needed to reach all states — call this number k. On the k^{th} iteration, T will detect that every state in M has been visited and will accept. Thus, $\langle M \rangle \in L(T)$.

2. USELESS is coRE-hard. To show this we prove that $\overline{\mathrm{A_{TM}}} \leq_m$ USELESS. Then as $\overline{\mathrm{A_{TM}}}$ is coRE-hard, so is USELESS from reduction transitivity.

   - Construction: The reduction TM M_f works as follows. Given input $\langle M, w \rangle$, M_f constructs the TM H that works as follows. On input x, H deletes x, writes w, runs M on w. If M accepts w, H moves to a new state (**which isn't in the original TM M!**), from which it traverses every state of itself, and then accepts. If M rejects w, then H rejects.
   - Correctness: If $\langle M, w \rangle \in \overline{\mathrm{A_{TM}}}$, then M does not accept w. Thus, H never reaches the special traversing state, so $\langle H \rangle \in$ USELESS. On the other hand, if $\langle M, w \rangle \notin \overline{\mathrm{A_{TM}}}$, then M accepts w, so H always reaches the traversal state, and thus visits every state in the machine. So $\langle H \rangle \notin$ USELESS.
   - Computability: Here we face a gap that was left from the construction: how to construct a traversing-state? It's not trivial, and it's the sort of thing we must explain in order for this answer to be correct.

     The idea is to write on the tape a special symbol @ on 2 cells and from every state in H make a transition to a "next" state upon reading @ traversing left and right over the @ symbols (having ordered the states arbitrarily). (The traversing-state writes this special symbol on the tape, and starts the traversal.)

     This promises that every state is visited, but that it won't be used before we get to the special state.

Finally, we remark that the rest of the construction is computable — the simulation part is as we have seen in class, and the traversal part is easy to compute — simply order the states of the machine and add the appropriate transitions.

□

Example: Languages hard in both classes: We have previously shown that $\text{HALT}_{\text{TM}}, \overline{\text{HALT}_{\text{TM}}} \leq_m \text{ALL}_{\text{TM}}$, so ALL_{TM} is both RE-hard and coRE-hard. Similarly, REG_{TM} is hard in both classes.

Question: What about R-hardness and completeness? In the exercise you will characterize those languages.

2 Busy Beaver Problem

The Busy Beaver problem is defined as follows:
Given some n, for all TMs with n states, and binary alphabet, evaluate the maximum number of steps till all TMs halt on the empty word ϵ, excluding TMs that do not halt at all over ϵ. For $n = 1$ there are 25 distinct TMs, $n = 2 \implies$ 6000+, $n = 3 \implies$ 4 million+.
The Busy Beaver is the machine that halts with maximum steps denoted as $BB(n)$.

Proposition 9. *$BB(n)$ is incomputable.*

Remark: This is not a decision problem, the question is whether there exists a function that can calculate this number.

Remark: The proof that HALT_ϵ with binary alphabet also belongs to $\text{RE} \setminus \text{R}$ is left as an exercise.

Proof. Assume by contradiction that it is computable, we show that HALT_ϵ is decidable.
A TM T that decides HALT_ϵ works as follows:

- Given $\langle M \rangle$, T computes $BB(|M|)$ and get some number t.
- T runs M on ϵ for t steps. If it halts, accept, otherwise reject.

T decides HALT_ϵ. If $\langle M \rangle \in \text{HALT}_\epsilon$, since the longest non-looping machine must halt after $BB(|M|)$ steps, then necessarily M will halt after this amount of steps, with the run of M over ϵ. (If we were to suppose that it took more steps that would be a contradiction to BB which we assumed finds the maximum halting size).
If $\langle M \rangle \notin \text{HALT}_\epsilon$ then M does not halt on ϵ, as such T rejects after t steps. So $\langle M \rangle \notin L(T)$.
Notice that BB provides a finite value, so T always halts, and T decides HALT_ϵ - contradiction. □

Remark: In 1989 Marxen and Buntrock discovered a 5 state machine that halts after 47,176,870 steps. It was only recently proven that this indeed is the value of $BB(5)$.